

Encontro 12: Boas Práticas de Programação

Sumário

Encontro 12: Boas Práticas de Programação	1
1. Introdução às Boas Práticas de Programação	3
1.1 O que são Boas Práticas de Programação?	3
1.2 Importância no Mercado de Trabalho	3
1.3 Impacto na Qualidade do Software	3
2. Princípios Fundamentais de Código Limpo	3
2.1 Legibilidade e Clareza	3
2.2 Simplicidade e Minimalismo	4
2.3 Consistência	4
3. Convenções de Nomenclatura	5
3.1 Convenções por Tecnologia	5
3.2 Nomes Expressivos e Significativos	6
3.3 Consistência em Projetos	7
4. Estruturação e Organização de Código	7
4.1 Arquitetura de Projetos Web	7
4.2 Modularização e Separação de Responsabilidades	8
4.3 Padrões de Design e Arquitetura	10
5. Comentários e Documentação	11
5.1 Comentários Eficazes	12
5.2 Documentação de Código (JSDoc)	12
5.3 Documentação de APIs	14
5.4 README e Documentação de Projeto	15
6. Tratamento de Erros e Debugging	16
6.1 Estratégias de Tratamento de Erros	16
6.2 Logging e Monitoramento	17
6.3 Ferramentas de Debugging	18
7. Versionamento e Controle de Código	18
7.1 Git e Controle de Versão	19

7.2 Estratégias de Branching.....	19
7.3 Conventional Commits	20
8. Testes e Qualidade de Código	20
8.1 Tipos de Testes.....	21
8.2 Ferramentas de Qualidade	23
8.3 Cobertura de Código e Métricas	24
9. Performance e Otimização	24
9.1 Otimização de Front-end.....	25
9.2 Otimização de Back-end	26
9.3 Monitoramento e Métricas	27
10. Segurança em Desenvolvimento Web	28
10.1 Autenticação e Autorização	28
10.2 Proteção Contra Vulnerabilidades.....	30
10.3 Headers de Segurança.....	31
11. Exercícios Práticos.....	32
11.1 Exercício 1: Refatoração de Código Legado.....	32
11.2 Exercício 2: Sistema de Autenticação Seguro (60 minutos)	36
11.3 Exercício 3: API RESTful com Testes (75 minutos)	38
11.4 Exercício 4: Aplicação Full-Stack com CI/CD (90 minutos)	41
12. Referências.....	43
12.1 Livros e Publicações	43
12.2 Documentação Oficial	43
12.3 Ferramentas e Recursos	44
12.4 Comunidades e Blogs	44

1. Introdução às Boas Práticas de Programação

1.1 O que são Boas Práticas de Programação?

As boas práticas de programação representam um conjunto de diretrizes, convenções e metodologias que visam melhorar a qualidade, legibilidade, manutenibilidade e eficiência do código. No contexto do desenvolvimento web moderno, essas práticas são fundamentais para criar aplicações robustas, escaláveis e sustentáveis ao longo do tempo [1] (vide referências).

O desenvolvimento web contemporâneo envolve múltiplas tecnologias, desde front-end com HTML, CSS e JavaScript até back-end com diversas linguagens e frameworks. Cada camada da aplicação web possui suas próprias especificidades, mas todas compartilham princípios fundamentais que garantem a qualidade do software desenvolvido.

1.2 Importância no Mercado de Trabalho

No mercado de trabalho atual, a capacidade de escrever código limpo e seguir boas práticas é um diferencial competitivo essencial. Empresas de tecnologia valorizam profissionais que não apenas resolvem problemas, mas o fazem de maneira elegante, eficiente e sustentável. Estudos indicam que desenvolvedores passam aproximadamente 80% do tempo lendo e compreendendo código existente, e apenas 20% escrevendo código novo [2].

A adoção de boas práticas resulta em benefícios tangíveis para organizações, incluindo redução de custos de manutenção, diminuição de bugs em produção, facilidade de onboarding de novos desenvolvedores e maior velocidade de desenvolvimento de novas funcionalidades. Projetos que seguem boas práticas desde o início tendem a ter ciclos de vida mais longos e maior retorno sobre investimento.

1.3 Impacto na Qualidade do Software

A qualidade do software é diretamente influenciada pela aplicação consistente de boas práticas de programação. Código bem estruturado e documentado reduz significativamente a incidência de bugs, facilita a identificação e correção de problemas, e permite que equipes trabalhem de forma mais colaborativa e eficiente.

Métricas de qualidade como complexidade ciclomática, cobertura de testes, duplicação de código e aderência a padrões de codificação são indicadores diretos da aplicação de boas práticas. Ferramentas modernas de análise estática de código, como SonarQube, ESLint e Prettier, automatizam a verificação dessas métricas e ajudam equipes a manter altos padrões de qualidade.

2. Princípios Fundamentais de Código Limpo

2.1 Legibilidade e Clareza

A legibilidade é o princípio mais básico do código limpo. Código legível é aquele que pode ser entendido rapidamente por qualquer desenvolvedor, independentemente de ter sido o autor original. Este princípio se manifesta por meio de várias práticas específicas que devem ser aplicadas consistentemente em projetos de desenvolvimento web.

No contexto do front-end, a legibilidade se aplica tanto ao HTML semântico quanto ao CSS bem estruturado e ao JavaScript expressivo. HTML semântico utiliza elementos apropriados para seu propósito, como `<header>`, `<nav>`, `<main>`, `<article>`, e `<footer>`, facilitando a compreensão da estrutura da página tanto para desenvolvedores quanto para tecnologias assistivas.

CSS legível emprega seletores descritivos, organização lógica de propriedades e uso consistente de metodologias como BEM (Block Element Modifier) ou SMACSS (Scalable and Modular Architecture for CSS). JavaScript legível prioriza funções pequenas e focadas, nomes de variáveis expressivos e estruturas de controle claras.

No back-end, a legibilidade se traduz em APIs bem documentadas, estruturas de dados claras e lógica de negócio bem-organizada. Independentemente da linguagem utilizada (Node.js, Python, PHP, Java), os princípios de legibilidade permanecem consistentes: nomes significativos, funções com responsabilidade única e estruturas de código que refletem o domínio do problema.

2.2 Simplicidade e Minimalismo

O princípio KISS (Keep It Simple, Stupid) deve ser sempre aplicado no desenvolvimento web. Simplicidade não significa funcionalidade limitada, mas sim soluções elegantes que resolvem problemas complexos de maneira direta e compreensível. No desenvolvimento web, isso se manifesta através da escolha de tecnologias apropriadas, arquiteturas não excessivamente complexas e implementações diretas.

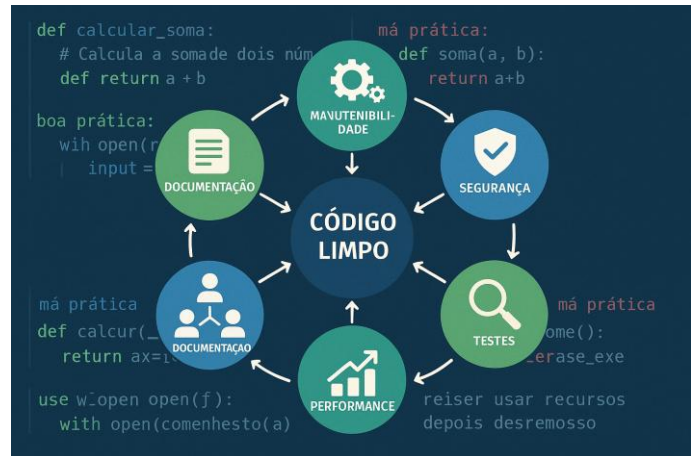
No front-end, simplicidade significa evitar over-engineering em componentes, utilizar CSS vanilla quando frameworks não são necessários, e implementar JavaScript progressivo que funciona mesmo quando recursos avançados não estão disponíveis. A regra dos três cliques para navegação e interfaces intuitivas são exemplos de simplicidade aplicada à experiência do usuário.

No back-end, simplicidade se traduz em APIs RESTful bem definidas, estruturas de banco de dados normalizadas mas não excessivamente complexas, e arquiteturas que crescem com as necessidades do projeto sem adicionar complexidade desnecessária desde o início.

2.3 Consistência

Consistência é crucial para manter a qualidade do código em projetos de equipe e ao longo do tempo. Isso inclui convenções de nomenclatura, estruturas de arquivos, padrões de codificação e metodologias de desenvolvimento. Ferramentas como Prettier, ESLint, e StyleLint automatizam a aplicação de regras de consistência, reduzindo a carga cognitiva dos desenvolvedores.

A consistência se estende além do código para incluir convenções de commit, estruturas de branch, documentação de APIs e padrões de deploy. Projetos consistentes são mais fáceis de manter, têm menor curva de aprendizado para novos desenvolvedores e apresentam menos bugs relacionados a mal-entendidos ou inconsistências.



Descrição da Ilustração: Pilares das boas práticas de programação organizados em torno do conceito central de "Código Limpo". Os seis elementos interconectados são: Manutenibilidade, Segurança, Testes, Performance, Documentação e Documentação colaborativa. Ao fundo, exemplos de código mostram contrastes entre boas e más práticas, como funções bem nomeadas versus nomes genéricos, demonstrando visualmente a diferença na qualidade do código.

3. Convenções de Nomenclatura

Convenções de Nomenclatura			
	Boas Práticas	Más Práticas	
camelCase	getUserName ✓	getusername	cetusername
	calculateTotal	calculatetotal	calculatetotal
	JavaScript variável	javaScritavável	javascript id
PascalCase	UserProfile	Userprofile	Userprofile
	DatabaseConnector	Databaseconnectio	Databaseconnection
	ID HTML ID	ID HTML ID	ID X abase-rábria
snake_case	user_name ✓	username	getuserdata
	get_user_data	/getuserdata	/getuserdata
	Tabela DB banco	/API Endpoint	/API Endpoint
kebab-case	user-profile ✓	userprofile	userprofile
	main-content	main-content	maincontent
	CSS classe	API Endpoint	API Endpoint

Descrição da Ilustração: Principais convenções de nomenclatura utilizadas no desenvolvimento web, organizadas em quatro estilos: camelCase (para variáveis JavaScript), PascalCase (para classes e componentes), snake_case (para tabelas de banco de dados), e kebab-case (para classes CSS). Cada estilo é mostrado com exemplos de boas práticas (marcados com ✓) e más práticas, demonstrando a aplicação correta em diferentes contextos como JavaScript, HTML, CSS e APIs.

3.1 Convenções por Tecnologia

As convenções de nomenclatura variam conforme a tecnologia e o contexto de uso, mas devem ser aplicadas consistentemente dentro de cada domínio. No desenvolvimento web, diferentes camadas da

aplicação seguem convenções específicas que refletem as melhores práticas da comunidade e facilitam a colaboração entre desenvolvedores.

JavaScript e Front-end:

No JavaScript, a convenção camelCase é predominante para variáveis, funções e métodos. Nomes como `getUserData()`, `calculateTotalPrice()`, e `isUserAuthenticated` são exemplos de nomenclatura clara e expressiva. Para constantes, utiliza-se UPPER_SNAKE_CASE, como `MAX_RETRY_ATTEMPTS` ou `API_BASE_URL`.

Classes e construtores seguem PascalCase: `UserProfile`, `DatabaseConnection`, `PaymentProcessor`. Esta convenção facilita a distinção entre funções regulares e construtores, melhorando a legibilidade do código.

CSS e Estilização:

Para CSS, a convenção kebab-case é amplamente adotada: `.user-profile`, `.main-navigation`, `.product-card`. Metodologias como BEM (Block Element Modifier) estendem essa convenção: `.product-card__title`, `.product-card__price`, `.product-card--featured`.

Variáveis CSS customizadas (CSS Custom Properties) seguem a mesma convenção: `--primary-color`, `--font-size-large`, `--spacing-medium`. Esta consistência facilita a manutenção e compreensão dos estilos.

HTML e Semântica:

Atributos HTML como `id` e `class` seguem kebab-case: `<div id="main-content">`, `<section class="hero-section">`. Para atributos `data-*`, a convenção se mantém: `data-user-id`, `data-product-category`.

Back-end e Banco de Dados:

No back-end, as convenções variam conforme a linguagem, mas snake_case é comum para bancos de dados: `user_profiles`, `order_items`, `created_at`. APIs RESTful frequentemente utilizam kebab-case para endpoints: `/api/user-profiles`, `/api/order-history`.

3.2 Nomes Expressivos e Significativos

Nomes de variáveis, funções e classes devem expressar claramente sua intenção e propósito. Evitar abreviações desnecessárias e nomes genéricos como `data`, `info`, `temp` melhora significativamente a legibilidade do código.

Exemplos de Boas Práticas:

```
// Bom: expressivo e claro
const userAuthenticationToken = generateToken(user);
const isValidEmail = validateEmail(email);
const calculateShippingCost = (weight, distance) => { /* ... */ };

// Ruim: genérico e pouco expressivo
const token = gen(u);
const valid = val(e);
const calc = (w, d) => { /* ... */ };
```

Contexto e Domínio:

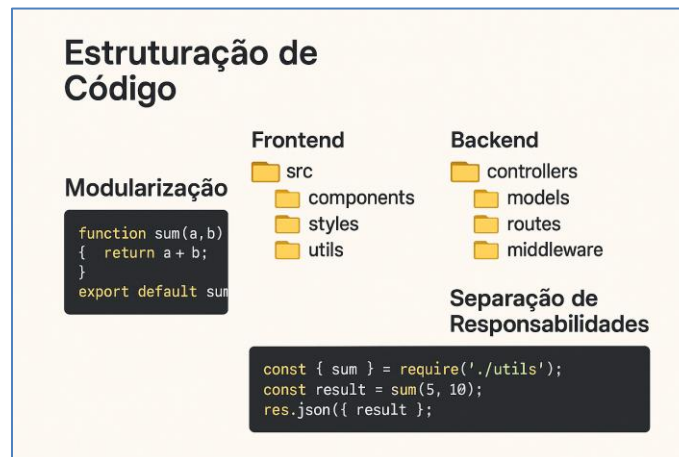
Nomes devem refletir o domínio do problema. Em um sistema de e-commerce, termos como `Product`, `Order`, `Customer`, `Inventory` são mais apropriados que termos genéricos. Esta prática facilita a comunicação entre desenvolvedores e stakeholders do negócio.

3.3 Consistência em Projetos

Estabelecer e manter convenções consistentes em todo o projeto é vital para a manutenibilidade. Ferramentas como ESLint, Prettier, e StyleLint podem automatizar a aplicação dessas convenções, reduzindo debates sobre estilo e focando a atenção em aspectos mais importantes do desenvolvimento.

Documentar as convenções adotadas em um guia de estilo do projeto garante que todos os membros da equipe sigam as mesmas regras. Este documento deve incluir exemplos práticos e justificativas para as escolhas feitas, facilitando a adoção e compreensão das convenções.

4. Estruturação e Organização de Código



Descrição da Ilustração: Organização básica de projetos web, dividida entre Frontend e Backend. No Frontend temos a estrutura de pastas típica (src, components, styles, utils), enquanto no Backend usamos a organização MVC (controllers, models, routes, middleware). A seção "Modularização" exibe um exemplo de função bem estruturada, e "Separação de Responsabilidades" mostra como importar e utilizar módulos de forma organizada, demonstrando as boas práticas de arquitetura de código.

4.1 Arquitetura de Projetos Web

A organização estrutural de projetos web garante a manutenibilidade e escalabilidade das aplicações. Uma arquitetura bem definida facilita a localização de código, reduz a duplicação e promove a reutilização de componentes. No desenvolvimento web moderno, a separação clara entre front-end e back-end, juntamente com a organização interna de cada camada, é essencial para projetos sustentáveis.

Estrutura de Front-end:

A organização do front-end deve refletir a funcionalidade e facilitar o desenvolvimento colaborativo. Uma estrutura típica inclui:

```
src/
├── components/           # Componentes reutilizáveis
│   ├── common/          # Componentes genéricos
│   ├── forms/           # Componentes de formulário
│   └── layout/          # Componentes de layout
├── pages/               # Páginas da aplicação
├── styles/              # Estilos globais e variáveis
│   ├── globals.css      # Estilos globais
│   ├── variables.css    # Variáveis CSS
│   └── components/      # Estilos específicos
├── utils/               # Funções utilitárias
├── services/            # Serviços de API
├── hooks/               # Custom hooks (React)
└── assets/              # Recursos estáticos
    ├── images/
    ├── icons/
    └── fonts/
```

Esta organização promove a separação de responsabilidades e facilita a manutenção. Componentes são agrupados por funcionalidade, estilos são organizados hierarquicamente e utilitários são centralizados para reutilização.

Estrutura de Back-end:

O back-end segue padrões arquiteturais estabelecidos, como MVC (Model-View-Controller) ou arquitetura em camadas:

```
src/
├── controllers/         # Lógica de controle
├── models/              # Modelos de dados
├── routes/              # Definição de rotas
├── middleware/          # Middleware personalizado
├── services/            # Lógica de negócio
├── utils/               # Funções utilitárias
├── config/              # Configurações
├── database/            # Configuração de banco
│   ├── migrations/     # Migrações
│   └── seeders/         # Dados iniciais
└── tests/              # Testes automatizados
```

4.2 Modularização e Separação de Responsabilidades

Modularização significa dividir o código em módulos menores, cada um com função específica. Este princípio tem o nome Single Responsibility Principle (SRP), base para criar código manutenível e testável.

Modularização no Front-end:

No JavaScript moderno, a modularização utiliza ES6 modules para criar componentes reutilizáveis:

```
// utils/validation.js
export const validateEmail = (email) => {
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  return emailRegex.test(email);
};
```

```
export const validatePassword = (password) => {
  return password.length >= 8 && /[A-Z]/.test(password) && /[0-9]/.test(password);
};
// components/LoginForm.js
import { validateEmail, validatePassword } from '../utils/validation.js';
export const LoginForm = () => {
  const handleSubmit = (formData) => {
    if (!validateEmail(formData.email)) {
      throw new Error('Email inválido');
    }
    if (!validatePassword(formData.password)) {
      throw new Error('Senha deve ter pelo menos 8 caracteres, uma maiúscula e um número');
    }
    // Processar login
  };
  return {
    handleSubmit,
    // Outros métodos do componente
  };
};
```

Modularização no Back-end:

No back-end, a modularização busca separar lógica de negócio, acesso a dados e controle de rotas:

```
// models/User.js
class User {
  constructor(userData) {
    this.id = userData.id;
    this.email = userData.email;
    this.name = userData.name;
  }
  static async findByEmail(email) {
    // Lógica de busca no banco de dados
  }
  async save() {
    // Lógica de persistência
  }
}
// services/AuthService.js
import { User } from '../models/User.js';
import { validateEmail, hashPassword } from '../utils/helpers.js';
export class AuthService {
  static async registerUser(userData) {
    if (!validateEmail(userData.email)) {
      throw new Error('Email inválido');
    }
    const existingUser = await User.findByEmail(userData.email);
    if (existingUser) {
      throw new Error('Usuário já existe');
    }
    const hashedPassword = await hashPassword(userData.password);
    const user = new User({ ...userData, password: hashedPassword });
    return await user.save();
  }
}
```

```
}  
// controllers/AuthController.js  
import { AuthService } from '../services/AuthService.js';  
export const registerController = async (req, res) => {  
  try {  
    const user = await AuthService.registerUser(req.body);  
    res.status(201).json({ user: user.toPublic() });  
  } catch (error) {  
    res.status(400).json({ error: error.message });  
  }  
};
```

4.3 Padrões de Design e Arquitetura

A aplicação de padrões de design estabelecidos melhora a qualidade e manutenibilidade do código. No desenvolvimento web, alguns padrões são particularmente relevantes:

Padrão MVC (Model-View-Controller):

Separa a aplicação em três camadas distintas: Model (dados), View (apresentação) e Controller (lógica de controle). Este padrão é muito usado em frameworks backend e facilita a manutenção e teste de aplicações.

Padrão Repository:

Abstrai o acesso a dados, permitindo que a lógica de negócio seja independente da implementação específica do banco de dados:

```
// repositories/UserRepository.js  
export class UserRepository {  
  async findById(id) {  
    // Implementação específica do banco  
  }  
  async findByEmail(email) {  
    // Implementação específica do banco  
  }  
  async save(user) {  
    // Implementação específica do banco  
  }  
}  
  
// services/UserService.js  
import { UserRepository } from '../repositories/UserRepository.js';  
export class UserService {  
  constructor(userRepository = new UserRepository()) {  
    this.userRepository = userRepository;  
  }  
  async getUserProfile(userId) {  
    const user = await this.userRepository.findById(userId);  
    if (!user) {  
      throw new Error('Usuário não encontrado');  
    }  
    return user.toPublic();  
  }  
}
```

Padrão Observer:

Útil para implementar sistemas de eventos e notificações, especialmente no front-end:

```
// utils/EventEmitter.js
export class EventEmitter {
  constructor() {
    this.events = {};
  }
  on(event, callback) {
    if (!this.events[event]) {
      this.events[event] = [];
    }
    this.events[event].push(callback);
  }
  emit(event, data) {
    if (this.events[event]) {
      this.events[event].forEach(callback => callback(data));
    }
  }
}

// Uso em componentes
const eventBus = new EventEmitter();
// Componente que emite eventos
eventBus.emit('user-login', { userId: 123, name: 'João' });
// Componente que escuta eventos
eventBus.on('user-login', (userData) => {
  console.log(`Usuário ${userData.name} fez login`);
});
```

5. Comentários e Documentação

ÚTEIS VS DESNECESSÁRIOS			
DESNECESSÁRIOS		COMENTÁRIOS	
// Obtém a idade a partir do ano de nascimento ✓ const idade = anoAtual - anoNascimento;		/* * Soma dois números * @ param {number} a 0 primeiro número * @ returns {number} A soma dos dois * @ example soma(2, 3); */	
✗ // Verifica se o usuario está autenticado if (usuario.autenticado) {		function soma(a, b) { return a + b; }	
✓ // Verifica se o usuário está autenticado if (usuário.autenticado) {			
JSDOC		README.md	
# Somucllo emeires		# Título do Projeto ## Instalação - Instruções de instalação ## Uso - Exemplos de uso	

Descrição da Ilustração: Os "Comentários Úteis" explicam o PORQUÊ da lógica (tentativa de reconexão quando a conexão é perdida), enquanto os "Comentários Inúteis" apenas repetem o que o código já expressa claramente. A seção JSDoc mostra documentação estruturada de função com parâmetros e descrições, e a "Documentação de API" apresenta um exemplo de documentação externa com autenticação e endpoints, demonstrando diferentes níveis de documentação necessários em projetos web.

5.1 Comentários Eficazes

Comentários eficazes explicam o "porquê" do código, não o "o quê". Código bem escrito deve ser autoexplicativo em relação ao que faz, mas comentários são valiosos para explicar decisões de design, contexto de negócio, ou lógica complexa que não é imediatamente óbvia.

Quando Comentar:

Comentários são mais úteis quando explicam:

- Decisões de design e trade-offs
- Lógica de negócio complexa
- Workarounds temporários
- Algoritmos não triviais
- Configurações específicas
- Limitações conhecidas

Exemplos de Comentários Úteis:

```
// Utilizamos debounce para evitar múltiplas chamadas à API
// durante a digitação rápida do usuário
const debouncedSearch = debounce(searchFunction, 300);
// Workaround: Safari não suporta CSS :focus-visible
// TODO: Remover quando o suporte for universal
if (isSafari) {
  element.classList.add('focus-fallback');
}
// Aplicamos retry exponential backoff para lidar com
// instabilidade temporária da API externa
const retryDelay = Math.pow(2, attemptNumber) * 1000;
```

Comentários a Evitar:

```
// Ruim: repete o que o código já diz
let userAge = 25; // Define a idade do usuário como 25
// Ruim: comentário óbvio
if (user.isActive) { // Se o usuário está ativo
  // Faz login
  performLogin();
}
// Ruim: comentário desatualizado
// Calcula o desconto de 10% (código agora calcula 15%)
const discount = price * 0.15;
```

5.2 Documentação de Código (JSDoc)

JSDoc é um padrão para documentar código JavaScript que permite gerar documentação automatizada. Esta prática é especialmente valiosa em projetos de equipe e bibliotecas reutilizáveis.

Estrutura Básica do JSDoc:

```
/**
 * Calcula o preço final com desconto e impostos aplicados
```

```
* @param {number} basePrice - Preço base do produto
* @param {number} discountPercent - Percentual de desconto (0-100)
* @param {number} taxRate - Taxa de imposto (0-1)
* @param {Object} options - Opções adicionais
* @param {boolean} options.roundToNearestCent - Arredondar para centavos
* @returns {number} Preço final calculado
* @throws {Error} Quando parâmetros são inválidos
* @example
* // Calcula preço com 10% de desconto e 8% de imposto
* const finalPrice = calculateFinalPrice(100, 10, 0.08);
* console.log(finalPrice); // 97.2
*/
function calculateFinalPrice(basePrice, discountPercent, taxRate, options = {}) {
  if (basePrice < 0 || discountPercent < 0 || discountPercent > 100) {
    throw new Error('Parâmetros inválidos');
  }
  const discountedPrice = basePrice * (1 - discountPercent / 100);
  const finalPrice = discountedPrice * (1 + taxRate);
  return options.roundToNearestCent
    ? Math.round(finalPrice * 100) / 100
    : finalPrice;
}
```

Documentação de Classes:

```
/**
 * Representa um usuário do sistema
 * @class
 */
class User {
  /**
   * Cria uma instância de User
   * @param {Object} userData - Dados do usuário
   * @param {string} userData.email - Email do usuário
   * @param {string} userData.name - Nome completo
   * @param {string} [userData.role='user'] - Papel do usuário no sistema
   */
  constructor(userData) {
    this.email = userData.email;
    this.name = userData.name;
    this.role = userData.role || 'user';
    this.createdAt = new Date();
  }
  /**
   * Verifica se o usuário tem permissão para uma ação
   * @param {string} action - Ação a ser verificada
   * @returns {boolean} True se o usuário tem permissão
   */
  hasPermission(action) {
    const permissions = this.getPermissions();
    return permissions.includes(action);
  }
}
```

5.3 Documentação de APIs

A documentação de APIs é requisito obrigatório para facilitar a integração e uso por outros desenvolvedores. Uma boa documentação de API inclui endpoints, parâmetros, exemplos de requisição e resposta, e códigos de erro.

Estrutura de Documentação de API:

API de Usuários

Autenticação

Todas as requisições devem incluir o header:
Authorization: Bearer {token}

Endpoints

GET /api/users

Retorna lista de usuários

****Parâmetros de Query:****

- `page` (number, opcional): Número da página (padrão: 1)
- `limit` (number, opcional): Itens por página (padrão: 10, máximo: 100)
- `search` (string, opcional): Busca por nome ou email

****Exemplo de Requisição:****

```
curl -H "Authorization: Bearer abc123" \
      "https://api.exemplo.com/users?page=1&limit=20&search=joão"
```

****Exemplo de Resposta (200 OK):****

```
{
  "data": [
    {
      "id": 1,
      "name": "João Silva",
      "email": "joao@exemplo.com",
      "role": "admin",
      "createdAt": "2024-01-15T10:30:00Z"
    }
  ],
  "pagination": {
    "page": 1,
    "limit": 20,
    "total": 150,
    "totalPages": 8
  }
}
```

****Códigos de Erro:****

- `400 Bad Request`: Parâmetros inválidos
- `401 Unauthorized`: Token inválido ou ausente
- `403 Forbidden`: Sem permissão para acessar
- `500 Internal Server Error`: Erro interno do servidor

Ferramentas de Documentação:

- **Swagger/OpenAPI:** Padrão para documentação de APIs REST
- **Postman:** Permite criar e compartilhar coleções de APIs
- **Insomnia:** Alternativa ao Postman com foco em simplicidade
- **GitBook:** Para documentação mais elaborada e guias

5.4 README e Documentação de Projeto

O arquivo README é frequentemente o primeiro contato que desenvolvedores têm com um projeto. Deve ser claro, conciso e fornecer as informações necessárias para começar a usar ou contribuir com o projeto.

Estrutura de README Eficaz:

```
# Nome do Projeto: _____
- Breve descrição do que o projeto faz e seu propósito.

## Funcionalidades:
- ✅ Autenticação de usuários
- ✅ CRUD de produtos
- ✅ Sistema de pagamentos
- 🚧 Relatórios avançados (em desenvolvimento)

## Tecnologias Utilizadas:
- Frontend: React 18, TypeScript, Tailwind CSS
- Backend: Node.js, Express, PostgreSQL
- Deploy: Docker, AWS

## Instalação
### Pré-requisitos:
- Node.js 18+
- PostgreSQL 14+
- Docker (opcional)

### Passos
1. Clone o repositório
   git clone https://github.com/usuario/projeto.git
   cd projeto
2. Instale as dependências
   npm install
3. Configure as variáveis de ambiente
   cp .env.example .env
Edite o arquivo .env com suas configurações
4. Execute as migrações
   npm run migrate
5. Inicie o servidor de desenvolvimento
   npm run dev

## Uso
Desenvolvimento
   npm run dev      # Inicia servidor de desenvolvimento
   npm run test     # Executa testes
   npm run build    # Gera build de produção
   npm run lint     # Verifica qualidade do código

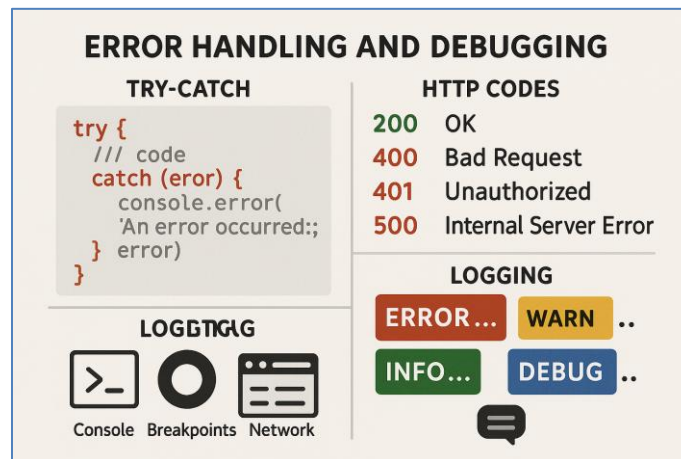
Produção
   Docker-compose up -d

## Contribuição
1. Faça um fork do projeto
2. Crie uma branch para sua feature (`git checkout -b feature/nova-feature`)
3. Commit suas mudanças (`git commit -am 'Adiciona nova feature'`)
4. Push para a branch (`git push origin feature/nova-feature`)
5. Abra um Pull Request

## Licença
Este projeto está sob a licença MIT. Veja o arquivo [LICENSE] (LICENSE) para detalhes.

## Contato
- Email: contato@projeto.com
- Slack: #projeto-dev
- Issues: [GitHub Issues] (https://github.com/usuario/projeto/issues)
```

6. Tratamento de Erros e Debugging



Descrição da Ilustração: Melhores práticas para tratamento de erros e debugging. Exemplos de try-catch em JavaScript, tratamento de erros em APIs REST com códigos HTTP apropriados, logging estruturado com diferentes níveis (error, warn, info, debug), e ferramentas de debugging como console.log, breakpoints e DevTools. Ênfase em mensagens de erro claras e logging adequado para a manutenção.

6.1 Estratégias de Tratamento de Erros

Tratar erros adequadamente é obrigatório para criar aplicações robustas e confiáveis. Diferentes tipos de erros requerem estratégias específicas, desde validação de entrada até falhas de rede e erros de servidor.

Front-end - JavaScript:

```
// Tratamento de erros síncronos  
try {  
  const result = JSON.parse(invalidJson);  
} catch (error) {  
  console.error('Erro ao parsear JSON:', error.message);  
  showUserFriendlyMessage('Dados inválidos recebidos');  
}  
  
// Tratamento de erros assíncronos  
async function fetchUserData(userId) {  
  try {  
    const response = await fetch(`/api/users/${userId}`);  
    if (!response.ok) {  
      throw new Error(`HTTP ${response.status}: ${response.statusText}`);  
    }  
    return await response.json();  
  } catch (error) {  
    if (error.name === 'TypeError') {  
      // Erro de rede  
      throw new Error('Falha na conexão. Verifique sua internet.');    }  
    // Re-throw outros erros  
    throw error;  
  }  
}  
  
// Global error handler
```

```
window.addEventListener('error', (event) => {
  console.error('Erro global capturado:', event.error);
  // Enviar erro para serviço de monitoramento
  errorReportingService.report(event.error);
});
```

Back-end - Node.js:

```
// Middleware de tratamento de erros
const errorHandler = (error, req, res, next) => {
  // Log do erro
  logger.error('Erro na aplicação:', {
    message: error.message,
    stack: error.stack,
    url: req.url,
    method: req.method,
    ip: req.ip
  });
  // Resposta baseada no tipo de erro
  if (error.name === 'ValidationError') {
    return res.status(400).json({
      error: 'Dados inválidos',
      details: error.details
    });
  }
  if (error.name === 'UnauthorizedError') {
    return res.status(401).json({
      error: 'Acesso não autorizado'
    });
  }
  // Erro genérico (não expor detalhes em produção)
  res.status(500).json({
    error: process.env.NODE_ENV === 'production'
      ? 'Erro interno do servidor'
      : error.message
  });
};
```

6.2 Logging e Monitoramento

Logging adequado é essencial para debugging e monitoramento de aplicações em produção. Logs devem ser estruturados, incluir contexto suficiente e usar níveis apropriados (error, warn, info, debug).

Logging Estruturado:

```
const logger = {
  error: (message, context = {}) => {
    console.error(JSON.stringify({
      level: 'error',
      message,
      timestamp: new Date().toISOString(),
      ...context
    }));
  },
  warn: (message, context = {}) => {
```

```
    console.warn(JSON.stringify({
      level: 'warn',
      message,
      timestamp: new Date().toISOString(),
      ...context
    }));
  },
  info: (message, context = {}) => {
    console.info(JSON.stringify({
      level: 'info',
      message,
      timestamp: new Date().toISOString(),
      ...context
    }));
  }
};
// Uso em contexto
logger.error('Falha ao processar pagamento', {
  userId: user.id,
  orderId: order.id,
  paymentMethod: 'credit_card',
  amount: order.total
});
```

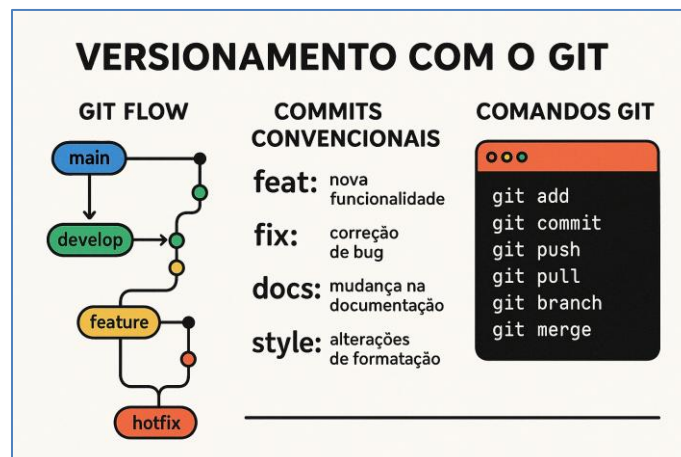
6.3 Ferramentas de Debugging

Ferramentas modernas de debugging facilitam a identificação e correção de problemas. Browser DevTools, debuggers de IDE, e ferramentas de profiling são essenciais para desenvolvimento eficiente.

Técnicas de Debugging:

- Uso estratégico de `console.log`, `console.table`, `console.group`
- Breakpoints condicionais em DevTools
- Network tab para debugging de APIs
- Performance tab para identificar gargalos
- Source maps para debugging de código transpilado

7. Versionamento e Controle de Código



Descrição da Ilustração: Melhores práticas de versionamento com Git, incluindo a estrutura de branches (main, develop, feature, hotfix), convenções de commit messages seguindo o padrão Conventional Commits, fluxo de trabalho GitFlow e comandos Git essenciais. Mostra como organizar branches, fazer merges e manter histórico limpo de commits, essencial para colaboração em equipe e manutenção de projetos.

7.1 Git e Controle de Versão

O Git é a ferramenta padrão para controle de versão no desenvolvimento web moderno. Compreender seus conceitos fundamentais e melhores práticas é essencial para trabalhar efetivamente em equipe e manter histórico organizado do projeto.

Conceitos Fundamentais:

- Repository (Repositório): Contém todo o histórico do projeto
- Commit: Snapshot do estado do código em um momento específico
- Branch: Linha de desenvolvimento independente
- Merge: Integração de mudanças entre branches
- Remote: Repositório remoto (GitHub, GitLab, Bitbucket)

Comandos Essenciais:

- a) Configuração inicial
`git init` # Inicializa repositório local
- b) Inicialização e clonagem
`git clone <url>` # Clona repositório remoto
- c) Operações básicas
`git add .` # Adiciona arquivos ao staging
`git commit -m "Mensagem descritiva"` # Cria commit
`git push origin main` # Envia commits para remoto
`git pull origin main` # Baixa atualizações do remoto
- d) Trabalho com branches
`git branch feature/nova-funcionalidade` # Cria nova branch
`git checkout feature/nova-funcionalidade` # Muda para branch
`git checkout -b feature/login` # Cria e muda para branch
`git merge feature/login` # Faz merge da branch
`git branch -d feature/login` # Deleta branch local

7.2 Estratégias de Branching

Estratégias de branching organizam o desenvolvimento e facilitam a colaboração. GitFlow e GitHub Flow são as abordagens mais populares, cada uma adequada para diferentes tipos de projeto.

GitFlow:

- main/master: Código em produção
- develop: Integração de features
- feature/*: Desenvolvimento de funcionalidades
- release/*: Preparação de releases
- hotfix/*: Correções urgentes em produção

GitHub Flow (Simplificado):

- main: Código sempre deployável
- feature/*: Branches de funcionalidade
- Pull Requests para revisão de código
- Deploy contínuo a partir da main

7.3 Conventional Commits

Conventional Commits padroniza mensagens de commit, facilitando automação e compreensão do histórico. A estrutura básica é: `tipo(escopo): descrição`.

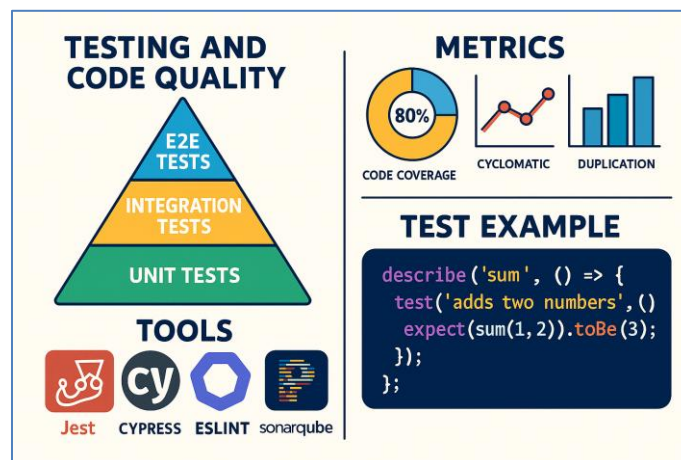
Tipos Comuns:

- feat: Nova funcionalidade
- fix: Correção de bug
- docs: Documentação
- style: Formatação (não afeta lógica)
- refactor: Refatoração de código
- test: Adição ou correção de testes
- chore: Tarefas de manutenção

Exemplos:

```
feat(auth): adiciona login com Google OAuth
fix(payment): corrige cálculo de juros compostos
docs(readme): atualiza instruções de instalação
style(css): formata arquivo de estilos
refactor(api): reorganiza estrutura de controllers
test(user): adiciona testes para validação de email
chore(deps): atualiza dependências do projeto
```

8. Testes e Qualidade de Código



Descrição da Ilustração: Pirâmide de testes (unitários, integração, e2e) e ferramentas de qualidade de código. Mostra exemplos de testes unitários com Jest, testes de integração para APIs, testes end-to-end com

Cypress, e ferramentas de análise estática como ESLint, Prettier e SonarQube. Ênfase na importância de diferentes tipos de testes e métricas de qualidade como cobertura de código e complexidade ciclomática.

8.1 Tipos de Testes

A estratégia de testes deve seguir a pirâmide de testes: muitos testes unitários, alguns testes de integração, e poucos testes end-to-end. Cada tipo serve a propósitos específicos e contribui para a confiabilidade da aplicação.

Testes Unitários:

Testam unidades isoladas de código (funções, métodos, componentes). São rápidos, focados e fáceis de manter.

```
// utils/validation.js
export function validateEmail(email) {
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  return emailRegex.test(email);
}

// tests/validation.test.js
import { validateEmail } from '../utils/validation.js';
describe('validateEmail', () => {
  test('deve retornar true para email válido', () => {
    expect(validateEmail('usuario@exemplo.com')).toBe(true);
  });
  test('deve retornar false para email inválido', () => {
    expect(validateEmail('email-invalido')).toBe(false);
    expect(validateEmail('')).toBe(false);
    expect(validateEmail('usuario@')).toBe(false);
  });
  test('deve lidar com casos extremos', () => {
    expect(validateEmail(null)).toBe(false);
    expect(validateEmail(undefined)).toBe(false);
  });
});
```

Testes de Integração:

Testam a interação entre diferentes módulos ou componentes. Verificam se as partes do sistema funcionam corretamente quando integradas.

```
// tests/api.integration.test.js
import request from 'supertest';
import app from '../src/app.js';
describe('API de Usuários', () => {
  test('POST /api/users deve criar usuário válido', async () => {
    const userData = {
      name: 'João Silva',
      email: 'joao@exemplo.com',
      password: 'senha123'
    };
    const response = await request(app)
      .post('/api/users')
```

```
        .send(userData)
        .expect(201);
    expect(response.body).toHaveProperty('id');
    expect(response.body.email).toBe(userData.email);
    expect(response.body).not.toHaveProperty('password');
  });
  test('POST /api/users deve rejeitar email duplicado', async () => {
    const userData = {
      name: 'Maria Silva',
      email: 'joao@exemplo.com', // Email já usado no teste anterior
      password: 'senha456'
    };
    await request(app)
      .post('/api/users')
      .send(userData)
      .expect(409);
  });
});
```

Testes End-to-End (E2E):

Testam fluxos completos da aplicação do ponto de vista do usuário. Simulam interações reais e verificam se a aplicação funciona como esperado.

```
// cypress/integration/login.spec.js
describe('Fluxo de Login', () => {
  it('deve permitir login com credenciais válidas', () => {
    cy.visit('/login');
    cy.get('[data-testid="email-input"]')
      .type('usuario@exemplo.com');
    cy.get('[data-testid="password-input"]')
      .type('senha123');
    cy.get('[data-testid="login-button"]')
      .click();
    cy.url().should('include', '/dashboard');
    cy.get('[data-testid="welcome-message"]')
      .should('contain', 'Bem-vindo');
  });
  it('deve mostrar erro para credenciais inválidas', () => {
    cy.visit('/login');
    cy.get('[data-testid="email-input"]')
      .type('usuario@exemplo.com');
    cy.get('[data-testid="password-input"]')
      .type('senha-errada');
    cy.get('[data-testid="login-button"]')
      .click();
    cy.get('[data-testid="error-message"]')
      .should('be.visible')
      .and('contain', 'Credenciais inválidas');
  });
});
```

8.2 Ferramentas de Qualidade

Ferramentas automatizadas de qualidade ajudam a manter padrões consistentes e identificar problemas potenciais antes que cheguem à produção.

ESLint - Análise Estática:

```
// .eslintrc.json
{
  "extends": ["eslint:recommended", "@typescript-eslint/recommended"],
  "rules": {
    "no-console": "warn",
    "no-unused-vars": "error",
    "prefer-const": "error",
    "no-var": "error",
    "complexity": ["error", 10],
    "max-lines-per-function": ["error", 50]
  }
}
```

Prettier - Formatação:

```
// .prettierrc
{
  "semi": true,
  "trailingComma": "es5",
  "singleQuote": true,
  "printWidth": 80,
  "tabWidth": 2
}
```

Husky - Git Hooks:

```
// package.json
{
  "husky": {
    "hooks": {
      "pre-commit": "lint-staged",
      "pre-push": "npm test"
    }
  },
  "lint-staged": {
    "*.{js,jsx,ts,tsx}": ["eslint --fix", "prettier --write"],
    "*.{css,scss}": ["stylelint --fix", "prettier --write"]
  }
}
```

8.3 Cobertura de Código e Métricas

Métricas de qualidade fornecem insights objetivos sobre a saúde do código. Cobertura de testes, complexidade ciclomática, e duplicação de código são indicadores importantes.

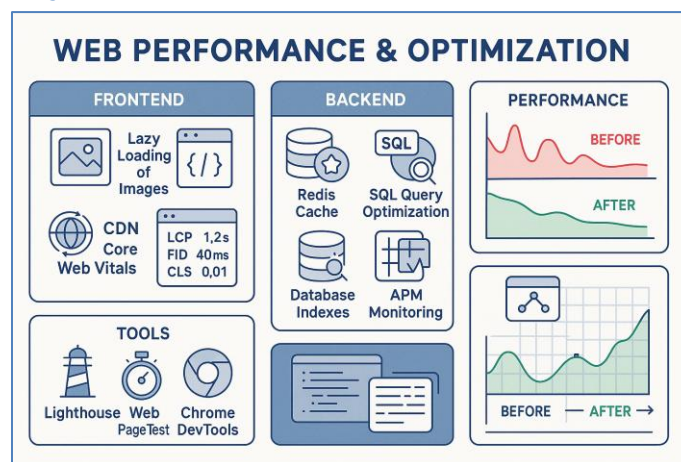
Configuração de Cobertura:

```
// jest.config.js
module.exports = {
  collectCoverage: true,
  coverageDirectory: 'coverage',
  coverageReporters: ['text', 'lcov', 'html'],
  coverageThreshold: {
    global: {
      branches: 80,
      functions: 80,
      lines: 80,
      statements: 80
    }
  }
};
```

Métricas Importantes:

- Cobertura de Linhas: Percentual de linhas executadas pelos testes
- Cobertura de Branches: Percentual de caminhos de código testados
- Complexidade Ciclomática: Medida da complexidade do código
- Duplicação: Percentual de código duplicado
- Débito Técnico: Estimativa de tempo para corrigir problemas

9. Performance e Otimização



Descrição da Ilustração: Técnicas de otimização de performance para aplicações web, dividida entre Frontend e Backend. No Frontend, mostra otimizações como lazy loading, minificação, compressão de imagens, e uso de CDN. No Backend, apresenta caching, otimização de queries de banco de dados, e monitoramento de performance. Métricas importantes: Core Web Vitals (LCP, FID, CLS) e ferramentas de análise como Lighthouse e WebPageTest.

9.1 Otimização de Front-end

A performance do front-end impacta diretamente a experiência do usuário e métricas de negócio. Core Web Vitals (LCP, FID, CLS) são métricas essenciais que devem ser monitoradas e otimizadas.

Otimização de Recursos:

```
// Lazy loading de imagens
const images = document.querySelectorAll('img[data-src]');
const imageObserver = new IntersectionObserver((entries, observer) => {
  entries.forEach(entry => {
    if (entry.isIntersecting) {
      const img = entry.target;
      img.src = img.dataset.src;
      img.classList.remove('lazy');
      observer.unobserve(img);
    }
  });
});
images.forEach(img => imageObserver.observe(img));
// Code splitting com dynamic imports
const loadComponent = async () => {
  const { default: Component } = await import('./HeavyComponent.js');
  return Component;
};
// Service Worker para caching
self.addEventListener('fetch', event => {
  if (event.request.destination === 'image') {
    event.respondWith(
      caches.open('images-v1').then(cache => {
        return cache.match(event.request).then(response => {
          return response || fetch(event.request).then(fetchResponse => {
            cache.put(event.request, fetchResponse.clone());
            return fetchResponse;
          });
        });
      })
    );
  }
});
```

Otimização de CSS:

```
/* Critical CSS inline no <head> */
/* Estilos above-the-fold */
/* CSS não crítico carregado assincronamente */
<link rel="preload" href="styles.css" as="style"
onload="this.onload=null;this.rel='stylesheet'">
/* Otimização de fontes */
@font-face {
  font-family: 'CustomFont';
  src: url('font.woff2') format('woff2');
  font-display: swap; /* Evita FOIT */
}
/* Uso eficiente de CSS Grid e Flexbox */
```

```
.container {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(300px, 1fr));
  gap: 1rem;
  contain: layout; /* CSS Containment */
}
```

9.2 Otimização de Back-end

Performance do back-end afeta tempo de resposta, escalabilidade e custos de infraestrutura. Otimização de queries, caching e arquitetura são fundamentais.

Otimização de Banco de Dados:

```
-- Índices apropriados
CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_orders_user_date ON orders(user_id, created_at);
-- Query otimizada com JOIN eficiente
SELECT u.name, COUNT(o.id) as order_count
FROM users u
LEFT JOIN orders o ON u.id = o.user_id
WHERE u.created_at >= '2024-01-01'
GROUP BY u.id, u.name
HAVING COUNT(o.id) > 5;
-- Paginação eficiente
SELECT * FROM products
WHERE id > :last_id
ORDER BY id
LIMIT 20;
```

Caching Estratégico:

```
// Redis para cache de sessões e dados frequentes
const redis = require('redis');
const client = redis.createClient();
// Cache de resultados de API
const getCacheData = async (key) => {
  try {
    const cached = await client.get(key);
    if (cached) {
      return JSON.parse(cached);
    }
    const data = await fetchFromDatabase(key);
    await client.setex(key, 3600, JSON.stringify(data)); // Cache por 1 hora
    return data;
  } catch (error) {
    console.error('Cache error:', error);
    return await fetchFromDatabase(key); // Fallback
  }
};
// Cache de HTTP com headers apropriados
app.get('/api/products', (req, res) => {
  res.set({
    'Cache-Control': 'public, max-age=300', // 5 minutos
    'ETag': generateETag(products),
  });
});
```

```
    'Last-Modified': products.lastModified
  });
  res.json(products);
});
```

9.3 Monitoramento e Métricas

Monitoramento contínuo permite identificar gargalos e regressões de performance. Ferramentas de APM (Application Performance Monitoring) são essenciais para aplicações em produção.

Métricas de Front-end:

```
// Core Web Vitals
import { getCLS, getFID, getFCP, getLCP, getTTFB } from 'web-vitals';
getCLS(console.log);
getFID(console.log);
getFCP(console.log);
getLCP(console.log);
getTTFB(console.log);
// Performance API
const observer = new PerformanceObserver((list) => {
  for (const entry of list.getEntries()) {
    if (entry.entryType === 'navigation') {
      console.log('Page Load Time:', entry.loadEventEnd - entry.fetchStart);
    }
  }
});
observer.observe({ entryTypes: ['navigation'] });
```

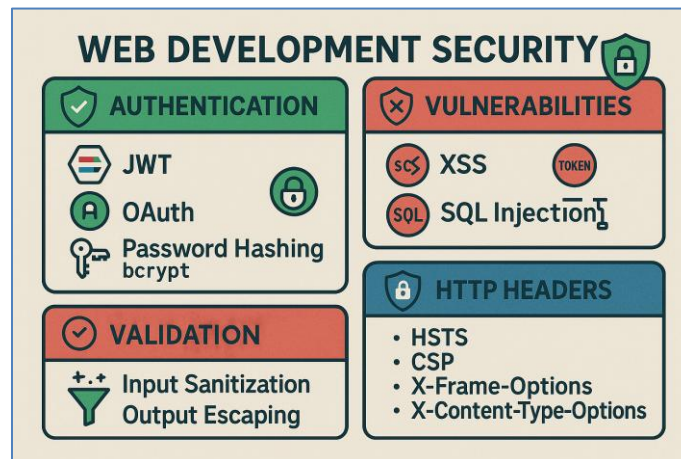
Métricas de Back-end:

```
// Middleware de timing
const responseTime = (req, res, next) => {
  const start = Date.now();
  res.on('finish', () => {
    const duration = Date.now() - start;
    console.log(`${req.method} ${req.path} - ${duration}ms`);
    // Enviar métricas para serviço de monitoramento
    metrics.timing('http.request.duration', duration, {
      method: req.method,
      route: req.route?.path,
      status: res.statusCode
    });
  });
  next();
};

// Health check endpoint
app.get('/health', async (req, res) => {
  const checks = {
    database: await checkDatabase(),
    redis: await checkRedis(),
    memory: process.memoryUsage(),
    uptime: process.uptime()
  };
  const isHealthy = Object.values(checks).every(check =>
```

```
    typeof check === 'object' ? check.status === 'ok' : check
  );
  res.status(isHealthy ? 200 : 503).json(checks);
});
```

10. Segurança em Desenvolvimento Web



Descrição da Ilustração: Principais práticas de segurança em desenvolvimento web, incluindo autenticação e autorização (JWT, OAuth), proteção contra vulnerabilidades comuns (XSS, CSRF, SQL Injection), validação de entrada, sanitização de dados e configuração segura de headers HTTP. A ilustração mostra exemplos de código seguro e inseguro, demonstrando como implementar medidas de proteção adequadas tanto no frontend quanto no backend.

10.1 Autenticação e Autorização

Segurança de autenticação e autorização é fundamental para proteger dados e funcionalidades. Implementação adequada previne acesso não autorizado e garante que usuários tenham apenas as permissões necessárias.

JWT (JSON Web Tokens):

```
// Geração de token seguro
const jwt = require('jsonwebtoken');
const bcrypt = require('bcrypt');
const generateToken = (user) => {
  const payload = {
    id: user.id,
    email: user.email,
    role: user.role
  };
  return jwt.sign(payload, process.env.JWT_SECRET, {
    expiresIn: '1h',
    issuer: 'myapp',
    audience: 'myapp-users'
  });
};
// Middleware de autenticação
const authenticateToken = (req, res, next) => {
```

```
const authHeader = req.headers['authorization'];
const token = authHeader && authHeader.split(' ')[1];
if (!token) {
  return res.status(401).json({ error: 'Token de acesso requerido' });
}
jwt.verify(token, process.env.JWT_SECRET, (err, user) => {
  if (err) {
    return res.status(403).json({ error: 'Token inválido' });
  }
  req.user = user;
  next();
});
};
// Middleware de autorização
const requireRole = (roles) => {
  return (req, res, next) => {
    if (!roles.includes(req.user.role)) {
      return res.status(403).json({ error: 'Permissão insuficiente' });
    }
    next();
  };
};
```

Hash de Senhas:

```
// Hash seguro de senhas
const hashPassword = async (password) => {
  const saltRounds = 12;
  return await bcrypt.hash(password, saltRounds);
};
// Verificação de senha
const verifyPassword = async (password, hash) => {
  return await bcrypt.compare(password, hash);
};
// Registro de usuário
app.post('/api/register', async (req, res) => {
  try {
    const { email, password } = req.body;
    // Validação de força da senha
    if (password.length < 8 || ![A-Z]/.test(password) || ![0-9]/.test(password)) {
      return res.status(400).json({
        error: 'Senha deve ter pelo menos 8 caracteres, uma maiúscula e um número'
      });
    }
    const hashedPassword = await hashPassword(password);
    const user = await User.create({ email, password: hashedPassword });
    res.status(201).json({ id: user.id, email: user.email });
  } catch (error) {
    res.status(500).json({ error: 'Erro interno do servidor' });
  }
});
```

10.2 Proteção Contra Vulnerabilidades

Proteção contra vulnerabilidades comuns (OWASP Top 10) deve ser implementada sistematicamente. XSS, CSRF, SQL Injection e outras ameaças requerem medidas específicas de prevenção.

Prevenção de XSS:

```
// Sanitização de entrada
const DOMPurify = require('dompurify');
const { JSDOM } = require('jsdom');
const window = new JSDOM('').window;
const purify = DOMPurify(window);
const sanitizeHtml = (dirty) => {
  return purify.sanitize(dirty, {
    ALLOWED_TAGS: ['b', 'i', 'em', 'strong', 'p', 'br'],
    ALLOWED_ATTR: []
  });
};

// Escape de output
const escapeHtml = (unsafe) => {
  return unsafe
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;")
    .replace(/'/g, "&#039;");
};

// Content Security Policy
app.use((req, res, next) => {
  res.setHeader('Content-Security-Policy',
    "default-src 'self'; " +
    "script-src 'self' 'unsafe-inline'; " +
    "style-src 'self' 'unsafe-inline'; " +
    "img-src 'self' data: https;");
  next();
});
```

Prevenção de CSRF:

```
const csrf = require('csrf');
// Configuração CSRF
const csrfProtection = csrf({
  cookie: {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'strict'
  }
});

app.use(csrfProtection);
// Endpoint para obter token CSRF
app.get('/api/csrf-token', (req, res) => {
  res.json({ csrfToken: req.csrfToken() });
});

// Verificação automática em formulários
```

```
app.post('/api/transfer', (req, res) => {
  // Token CSRF verificado automaticamente pelo middleware
  // Processar transferência...
});
```

Prevenção de SQL Injection:

```
// Uso de prepared statements
const mysql = require('mysql2/promise');
// SEGURO: Prepared statement
const getUserById = async (id) => {
  const [rows] = await connection.execute(
    'SELECT * FROM users WHERE id = ?',
    [id]
  );
  return rows[0];
};
// INSEGURO: String concatenation (NÃO FAZER)
const unsafeQuery = async (id) => {
  const query = `SELECT * FROM users WHERE id = ${id}`; // Vulnerável!
  return await connection.query(query);
};
// Validação de entrada
const validateUserId = (id) => {
  const userId = parseInt(id, 10);
  if (isNaN(userId) || userId <= 0) {
    throw new Error('ID de usuário inválido');
  }
  return userId;
};
```

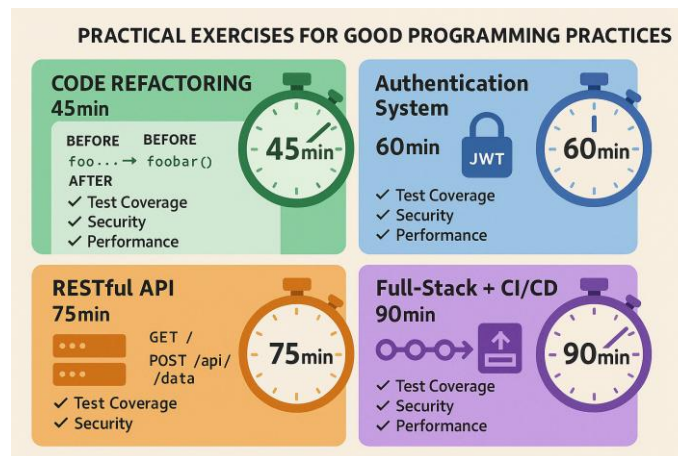
10.3 Headers de Segurança

Headers HTTP apropriados fornecem camadas adicionais de proteção contra diversos tipos de ataques. Configuração adequada é essencial para aplicações seguras.

```
const helmet = require('helmet');
// Configuração abrangente de headers de segurança
app.use(helmet({
  // Força HTTPS
  hsts: {
    maxAge: 31536000,
    includeSubDomains: true,
    preload: true
  },
  // Previne clickjacking
  frameguard: { action: 'deny' },
  // Previne MIME sniffing
  noSniff: true,
  // Força download de arquivos perigosos
  contentTypeOptions: false,
  // Remove header X-Powered-By
  hidePoweredBy: true,
  // Configuração CSP personalizada
  contentSecurityPolicy: {
```

```
directives: {
  defaultSrc: ['self'],
  styleSrc: ['self', 'unsafe-inline'],
  scriptSrc: ['self'],
  imgSrc: ['self', 'data:', 'https:'],
  connectSrc: ['self'],
  fontSrc: ['self'],
  objectSrc: ['none'],
  mediaSrc: ['self'],
  frameSrc: ['none']
}
});
// Headers customizados adicionais
app.use((req, res, next) => {
  // Previne cache de dados sensíveis
  res.setHeader('Cache-Control', 'no-store, no-cache, must-revalidate');
  res.setHeader('Pragma', 'no-cache');
  // Política de refererrer
  res.setHeader('Referrer-Policy', 'strict-origin-when-cross-origin');
  // Política de permissões
  res.setHeader('Permissions-Policy',
    'geolocation=(), microphone=(), camera=()');
  next();
});
```

11. Exercícios Práticos



Descrição da Ilustração: Visão geral dos exercícios propostos para consolidar o aprendizado de boas práticas de programação: (1) refatoração de código legado, (2) implementação de sistema de autenticação seguro, (3) criação de API RESTful com testes, e (4) desenvolvimento de aplicação full-stack com CI/CD.

11.1 Exercício 1: Refatoração de Código Legado

Objetivo: Aplicar princípios de código limpo para refatorar um código mal estruturado, melhorando legibilidade, manutenibilidade e testabilidade.

Cenário: Você recebeu um código JavaScript legado de um sistema de e-commerce que precisa ser refatorado. O código atual tem problemas de nomenclatura, estrutura e não segue boas práticas.

Código Inicial (Problemático):

```
// produto.js - CÓDIGO LEGADO PARA REFATORAR
var p = {
  n: '',
  pr: 0,
  d: '',
  c: '',
  s: 0
};
function calc(p, q, d) {
  var t = p * q;
  if (d > 0) {
    t = t - (t * d / 100);
  }
  return t;
}
function v(p) {
  if (p.n == '' || p.pr <= 0) {
    return false;
  }
  return true;
}
function save(p) {
  if (v(p)) {
    // salvar no banco
    console.log('salvando...');
    return true;
  } else {
    console.log('erro');
    return false;
  }
}
// uso
var produto = p;
produto.n = 'Notebook';
produto.pr = 1500;
produto.d = 'Notebook Dell';
produto.c = 'Eletrônicos';
produto.s = 10;
var total = calc(produto.pr, 2, 10);
console.log(total);
```

Tarefas a Realizar:

1. Nomenclatura Expressiva (10 min):

- Renomear variáveis e funções com nomes significativos
- Usar convenções apropriadas (camelCase)
- Eliminar abreviações desnecessárias

2. Estruturação e Modularização (15 min):

- Criar classe ou objeto bem estruturado para Product
- Separar responsabilidades em módulos distintos
- Implementar validações robustas

3. Documentação e Comentários (10 min):

- Adicionar JSDoc para funções públicas
- Incluir comentários explicativos onde necessário
- Criar README com instruções de uso

4. Tratamento de Erros (10 min):

- Implementar tratamento adequado de erros
- Usar try-catch onde apropriado
- Criar mensagens de erro informativas

Resultado Esperado:

```
/**
 * Representa um produto no sistema de e-commerce
 */
class Product {
  constructor(name, price, description, category, stock = 0) {
    this.name = name;
    this.price = price;
    this.description = description;
    this.category = category;
    this.stock = stock;
  }
  /**
   * Valida se o produto tem dados válidos
   * @returns {boolean} true se válido, false caso contrário
   */
  isValid() {
    return this.name.trim() !== '' &&
      this.price > 0 &&
      this.stock >= 0;
  }
  /**
   * Calcula o preço total com desconto aplicado
   * @param {number} quantity - Quantidade de produtos
   * @param {number} discountPercentage - Percentual de desconto (0-100)
   * @returns {number} Preço total com desconto
   */
  calculateTotalPrice(quantity, discountPercentage = 0) {
    if (quantity <= 0) {
      throw new Error('Quantidade deve ser maior que zero');
    }
    if (discountPercentage < 0 || discountPercentage > 100) {
      throw new Error('Desconto deve estar entre 0 e 100%');
    }
  }
}
```

```
    }
    const subtotal = this.price * quantity;
    const discountAmount = subtotal * (discountPercentage / 100);
    return subtotal - discountAmount;
  }
}
/**
 * Serviço para operações de produto
 */
class ProductService {
  /**
   * Salva um produto no sistema
   * @param {Product} product - Produto a ser salvo
   * @returns {Promise<boolean>} true se salvo com sucesso
   */
  async save(product) {
    try {
      if (!product.isValid()) {
        throw new Error('Produto com dados inválidos');
      }
      // Simular salvamento no banco de dados
      console.log(`Salvando produto: ${product.name}`);
      return true;
    } catch (error) {
      console.error('Erro ao salvar produto:', error.message);
      throw error;
    }
  }
}

// Exemplo de uso
const product = new Product(
  'Notebook Dell Inspiron',
  1500,
  'Notebook Dell com 8GB RAM e SSD 256GB',
  'Eletrônicos',
  10
);

const productService = new ProductService();
try {
  const totalPrice = product.calculateTotalPrice(2, 10);
  console.log(`Preço total com desconto: R$ ${totalPrice}`);
  await productService.save(product);
} catch (error) {
  console.error('Erro:', error.message);
}
```

11.2 Exercício 2: Sistema de Autenticação Seguro (60 minutos)

Objetivo: Implementar um sistema de autenticação completo seguindo melhores práticas de segurança, incluindo hash de senhas, JWT, e proteção contra vulnerabilidades.

Requisitos:

- Registro de usuários com validação de senha forte
- Login com JWT
- Middleware de autenticação
- Proteção contra ataques comuns
- Testes unitários básicos

Estrutura do Projeto:

```
auth-system/  
├── src/  
│   ├── models/  
│   │   └── User.js  
│   ├── services/  
│   │   └── AuthService.js  
│   ├── middleware/  
│   │   └── auth.js  
│   ├── routes/  
│   │   └── auth.js  
│   ├── utils/  
│   │   └── validation.js  
│   └── app.js  
├── tests/  
│   └── auth.test.js  
├── .env.example  
└── package.json
```

Implementação Base:

```
// src/models/User.js  
const bcrypt = require('bcrypt');  
class User {  
  constructor(id, email, password, createdAt = new Date()) {  
    this.id = id;  
    this.email = email;  
    this.password = password;  
    this.createdAt = createdAt;  
  }  
  static async hashPassword(password) {  
    const saltRounds = 12;  
    return await bcrypt.hash(password, saltRounds);  
  }  
  async verifyPassword(password) {  
    return await bcrypt.compare(password, this.password);  
  }  
  toJSON() {  
    const { password, ...userWithoutPassword } = this;
```

```
    return userWithoutPassword;
  }
}
module.exports = User;
// src/utils/validation.js
const validateEmail = (email) => {
  const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  return emailRegex.test(email);
};
const validatePassword = (password) => {
  // Pelo menos 8 caracteres, uma maiúscula, uma minúscula, um número
  const passwordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)[a-zA-Z\d@$!%*?&]{8,}$/;
  return passwordRegex.test(password);
};
module.exports = { validateEmail, validatePassword };
```

Tarefas a Implementar:

1. AuthService Completo (20 min):

- Método de registro com validações
- Método de login com geração de JWT
- Verificação de email duplicado
- Rate limiting básico

2. Middleware de Autenticação (15 min):

- Verificação de token JWT
- Extração de dados do usuário
- Tratamento de erros de autenticação

3. Rotas de API (15 min):

- POST /register
- POST /login
- GET /profile (protegida)
- Validação de entrada

4. Testes Unitários (10 min):

- Testes para validação de email/senha
- Testes para AuthService
- Testes para middleware de auth

11.3 Exercício 3: API RESTful com Testes (75 minutos)

Objetivo: Desenvolver uma API RESTful completa para gerenciamento de tarefas, incluindo CRUD operations, validações, testes automatizados e documentação.

Especificações da API:

Método	Endpoint	Descrição	Autenticação
POST	/api/auth/register	Registrar usuário	Não
POST	/api/auth/login	Login	Não
GET	/api/tasks	Listar tarefas	Sim
POST	/api/tasks	Criar tarefa	Sim
GET	/api/tasks/:id	Obter tarefa	Sim
PUT	/api/tasks/:id	Atualizar tarefa	Sim
DELETE	/api/tasks/:id	Deletar tarefa	Sim

Modelo de Dados:

```
// Task Model
{
  id: String,
  title: String (required, max 100 chars),
  description: String (optional, max 500 chars),
  status: Enum ['pending', 'in_progress', 'completed'],
  priority: Enum ['low', 'medium', 'high'],
  dueDate: Date (optional),
  userId: String (required),
  createdAt: Date,
  updatedAt: Date
}
```

Tarefas a Implementar:

1. Estrutura do Projeto (15 min):

- Configurar Express.js com middleware essencial
- Configurar banco de dados (MongoDB/SQLite)
- Estrutura de pastas organizada

2. Modelos e Validações (20 min):

- Modelo Task com validações
- Modelo User
- Schemas de validação com Joi/Yup

3. Controllers e Rotas (25 min):

- CRUD completo para tasks
- Filtros e paginação
- Tratamento de erros consistente

4. Testes de Integração (15 min):

- Testes para todos os endpoints
- Setup e teardown de banco de teste
- Assertions apropriadas

Exemplo de Implementação:

```
// src/controllers/TaskController.js
const Task = require('../models/Task');
const { validateTask } = require('../utils/validation');
class TaskController {
  async index(req, res) {
    try {
      const { page = 1, limit = 10, status, priority } = req.query;
      const userId = req.user.id;
      const filters = { userId };
      if (status) filters.status = status;
      if (priority) filters.priority = priority;
      const tasks = await Task.find(filters)
        .limit(limit * 1)
        .skip((page - 1) * limit)
        .sort({ createdAt: -1 });
      const total = await Task.countDocuments(filters);
      res.json({
        tasks,
        pagination: {
          page: parseInt(page),
          limit: parseInt(limit),
          total,
          pages: Math.ceil(total / limit)
        }
      });
    } catch (error) {
      res.status(500).json({ error: 'Erro interno do servidor' });
    }
  }
  async store(req, res) {
    try {
      const { error, value } = validateTask(req.body);
      if (error) {
        return res.status(400).json({
          error: 'Dados inválidos',
          details: error.details
        });
      }
      const task = new Task({
        ...value,
        userId: req.user.id
      });
      await task.save();
      res.status(201).json(task);
    } catch (error) {
      res.status(500).json({ error: 'Erro ao criar tarefa' });
    }
  }
}
```

```
    }
  }
  async show(req, res) {
    try {
      const task = await Task.findOne({
        _id: req.params.id,
        userId: req.user.id
      });
      if (!task) {
        return res.status(404).json({ error: 'Tarefa não encontrada' });
      }
      res.json(task);
    } catch (error) {
      res.status(500).json({ error: 'Erro interno do servidor' });
    }
  }
  async update(req, res) {
    try {
      const { error, value } = validateTask(req.body, { partial: true });
      if (error) {
        return res.status(400).json({
          error: 'Dados inválidos',
          details: error.details
        });
      }
      const task = await Task.findOneAndUpdate(
        { _id: req.params.id, userId: req.user.id },
        { ...value, updatedAt: new Date() },
        { new: true }
      );
      if (!task) {
        return res.status(404).json({ error: 'Tarefa não encontrada' });
      }
      res.json(task);
    } catch (error) {
      res.status(500).json({ error: 'Erro ao atualizar tarefa' });
    }
  }
  async destroy(req, res) {
    try {
      const task = await Task.findOneAndDelete({
        _id: req.params.id,
        userId: req.user.id
      });
      if (!task) {
        return res.status(404).json({ error: 'Tarefa não encontrada' });
      }
      res.status(204).send();
    } catch (error) {
      res.status(500).json({ error: 'Erro ao deletar tarefa' });
    }
  }
}
module.exports = new TaskController();
```

11.4 Exercício 4: Aplicação Full-Stack com CI/CD (90 minutos)

Objetivo: Desenvolver uma aplicação completa (front-end + back-end) com pipeline de CI/CD, demonstrando integração de todas as boas práticas abordadas.

Especificações do Projeto:

- Front-end: React.js com TypeScript
- Back-end: Node.js/Express com TypeScript
- Banco de Dados: PostgreSQL
- Autenticação: JWT
- Testes: Jest + Cypress
- CI/CD: GitHub Actions
- Deploy: Docker + Heroku/Vercel

Funcionalidades:

1. Sistema de autenticação completo
2. Dashboard com métricas
3. CRUD de usuários (admin)
4. CRUD de produtos
5. Sistema de notificações
6. Relatórios básicos

Estrutura do Projeto:

```
fullstack-app/  
├── frontend/  
│   ├── src/  
│   │   ├── components/  
│   │   ├── pages/  
│   │   ├── services/  
│   │   ├── hooks/  
│   │   └── utils/  
│   ├── public/  
│   ├── tests/  
│   └── package.json  
├── backend/  
│   ├── src/  
│   │   ├── controllers/  
│   │   ├── models/  
│   │   ├── routes/  
│   │   ├── middleware/  
│   │   └── utils/  
│   ├── tests/  
│   └── package.json  
├── docker-compose.yml  
├── .github/  
│   └── workflows/  
│       └── ci-cd.yml  
└── README.md
```

Pipeline CI/CD:

Arquivo `.github/workflows/ci-cd.yml`

```
name: CI/CD Pipeline
on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:13
        env:
          POSTGRES_PASSWORD: postgres
          POSTGRES_DB: test_db
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
    steps:
      - uses: actions/checkout@v2
      - name: Setup Node.js
        uses: actions/setup-node@v2
        with:
          node-version: '16'
          cache: 'npm'
      - name: Install dependencies
        run: |
          cd backend && npm ci
          cd ../frontend && npm ci
      - name: Run linting
        run: |
          cd backend && npm run lint
          cd ../frontend && npm run lint
      - name: Run backend tests
        run: cd backend && npm test
        env:
          DATABASE_URL: postgresql://postgres:postgres@localhost:5432/test_db
      - name: Run frontend tests
        run: cd frontend && npm test -- --coverage
      - name: Run E2E tests
        run: cd frontend && npm run test:e2e
  deploy:
    needs: test
    runs-on: ubuntu-latest
    if: github.ref == 'refs/heads/main'
    steps:
      - uses: actions/checkout@v2
      - name: Deploy to production
        run: |
          # Deploy backend to Heroku
          # Deploy frontend to Vercel
          echo "Deploying to production..."
```

12. Referências

12.1 Livros e Publicações

1. **Clean Code: A Handbook of Agile Software Craftsmanship** - Robert C. Martin

- Princípios fundamentais de código limpo; Técnicas de refatoração; Boas práticas de nomenclatura e estruturação

2. **The Pragmatic Programmer** - David Thomas e Andrew Hunt

- Filosofia de desenvolvimento pragmático; Técnicas de debugging e testing; Gestão de projetos de software

3. **You Don't Know JS (book series)** - Kyle Simpson

- JavaScript avançado e boas práticas; Conceitos fundamentais da linguagem; Padrões de design em JavaScript

4. **Web Performance in Action** - Jeremy Wagner

- Otimização de performance web; Técnicas de carregamento eficiente; Métricas e monitoramento

12.2 Documentação Oficial

1. **MDN Web Docs** - <https://developer.mozilla.org/>

- Documentação completa de HTML, CSS, JavaScript; Guias de boas práticas; Exemplos práticos e tutoriais

2. **Node.js Documentation** - <https://nodejs.org/docs/>

- API reference completa; Guias de segurança e performance; Melhores práticas para desenvolvimento

3. **React Documentation** - <https://reactjs.org/docs/>

- Conceitos fundamentais; Hooks e componentes funcionais; Padrões de desenvolvimento

4. **Express.js Guide** - <https://expressjs.com/>

- Middleware e roteamento; Segurança e performance; Estruturação de aplicações

12.3 Ferramentas e Recursos

1. Análise de Código:

- ESLint: <https://eslint.org/>
- Prettier: <https://prettier.io/>
- SonarQube: <https://www.sonarqube.org/>

2. Testes:

- Jest: <https://jestjs.io/>
- Cypress: <https://www.cypress.io/>
- Testing Library: <https://testing-library.com/>

3. Performance:

- Lighthouse: <https://developers.google.com/web/tools/lighthouse>
- WebPageTest: <https://www.webpagetest.org/>
- Chrome DevTools: <https://developers.google.com/web/tools/chrome-devtools>

4. Segurança:

- OWASP Top 10: <https://owasp.org/www-project-top-ten/>
- Helmet.js: <https://helmetjs.github.io/>
- Snyk: <https://snyk.io/>

12.4 Comunidades e Blogs

1. Stack Overflow - <https://stackoverflow.com/>

- Comunidade de desenvolvedores; Soluções para problemas comuns; Discussões sobre boas práticas

2. GitHub - <https://github.com/>

- Repositórios de código aberto; Exemplos de projetos bem estruturados; Contribuições para projetos

3. Dev.to - <https://dev.to/>

- Artigos sobre desenvolvimento; Tutoriais e guias práticos; Discussões da comunidade

4. CSS-Tricks - <https://css-tricks.com/>

- Técnicas avançadas de CSS; Tutoriais de front-end; Boas práticas de design